IS115 Project – Team Work Plan

Oct 29, 2026 · @jie min

0. Setup + Tasks 1–2: metric and baseline (Fri 30 Oct – Sun 1 Nov)

Start from the prof's original `project.zip`. By the end of this section you'll have the scorer written and the baseline measured on all 20 days. That gives you the number every later policy has to beat.

Step 1: Setup (both, 30 min)

1. Person A unzips the prof's `project.zip` and puts the `project/` folder in a shared place (a GitHub repo is best; Google Drive works). Person B joins it.
2. Each person opens `project/` in VS Code and opens a terminal **inside `project/`**.
3. Check that the simulator runs: `python3 -m simulator --policy solution.policies.baseline --data data --day day_001 --output results/baseline/day_001`
4. Each person reads `solution/policies/baseline.py` line by line until they can explain every line. Your own policies will be modified copies of it.
5. Open `results/baseline/day_001/log.txt` and scroll through it. Find a `hub_departure` and the `delivery` lines that follow it.

Step 2: Write down the metric (Person A, 30 min, in your notes)

This becomes report section 1. Write down:

1. **Metric:** on-time rate = parcels delivered inside their window ÷ **all** parcels that arrived that day.
2. **Why all parcels:** if you divided by delivered parcels only, delivering 1 parcel on time would score 100%. With all parcels, dispatching nothing scores 0%, so there's no trivial solution.
3. **Assumptions:** early delivery = not on time (customer may not be home); undelivered = fail.
4. **Supporting metrics** (help explain results): delivery rate = delivered ÷ all parcels; average lateness in minutes; counts of on-time, early, late and undelivered parcels; number of trips.

Step 3: Code the scorer in `solution/performance_monitor.py` (Person A, 2 h)

Only edit the `calculate_metrics` function. Delete the trip-counting example inside it and leave `main()` alone. Build it in this order:

1. **A helper to convert times.** Parcel times in the CSV are strings like `"14:30"`, but log times are minutes (870.0). Write a small function that turns `"14:30"` into 870.
2. **A dictionary of windows.** Loop over `parcels` and store each `parcel_id` → (window start, window end) in minutes. A dict means each later lookup is O(1).
3. **Loop over `events`.** Skip everything except `event_type == "delivery"` (and count `hub_departure` events for trips). For each delivery, look up its window and decide: early, on time, or late. If late, add the minutes late to a running total.
4. **Count undelivered** = total parcels − parcels delivered.
5. **Return a dictionary**, e.g. `{"on_time_rate": ..., "delivery_rate": ..., "avg_lateness_min": ..., "on_time": ..., "early": ..., "late": ..., "undelivered": ..., "trips": ...}`. Guard every division so it never divides by zero.

Check it's correct: run `python3 -m solution.performance_monitor --data data --day day_001 --log results/baseline/day_001/log.txt` and open `results/baseline/day_001/metrics.json`. For day 1 you should get **799 parcels, 213 on time, 136 late, 0 early, 450 undelivered, 20 trips**, giving on-time rate 0.2666. If your numbers differ, your time conversion or window check has a bug.

Step 4: Shortcut script `run_all.sh` (Person B, 30 min)

Typing 2 commands × 20 days by hand is slow and error-prone. Create a file `run_all.sh` in the `project/` folder that loops over day_001–day_020 and, for each day, runs the simulator and then the performance monitor. Take the policy name as an argument, so `bash run_all.sh baseline` runs the baseline and `bash run_all.sh urgency_first` runs a later policy. This is boilerplate, so the Gemini Gem can write it. Paste the prompt and response into your references.

Step 5: Results table `summarise.py` (Person B, 30 min)

Create `summarise.py` in `project/`. It should read every `results/<policy>/<day>/metrics.json`, write one big table to `results/summary.csv` (one row per policy per day), and print the average of each metric per policy. You'll use this table for the report. Also boilerplate, also fine for the Gem.

Step 6: Run the baseline on all 20 days (Person B, 10 min)

1. Run `bash run_all.sh baseline`, then `python3 summarise.py`.

2. You should get an average on-time rate of about **27.4%** (lowest day about 24.8%, highest about 31.2%), and delivery rate about 40.5%. Matching numbers mean Person A's scorer is correct.
3. Keep the whole `results/baseline/` folder. It has to be in the final submission.

Rule for the shared folder: each person only edits their own files. That way you never overwrite each other's work.

1. Person A: urgency-first policy (Sun 1 Nov – Fri 6 Nov)

The idea: the baseline ignores deadlines. Person A's policy should load the parcels that are **most urgent** first. Your file: `solution/policies/urgency_first.py`.

Step 1: Create the file (10 min)

1. In `solution/policies/`, copy `baseline.py` and rename the copy `urgency_first.py`.
2. Change the docstring at the top to describe your policy.
3. Run it once unchanged to make sure the copy works: `bash run_all.sh urgency_first`. It should score exactly the same as your baseline.

Step 2: Design it on paper first (1–2 h, no coding)

Answer these in writing. Your answers become the "Algorithmic approaches" section of the report.

1. **What does "urgent" mean?** Earliest window end? Least time left before the deadline? Write the rule as one sentence.
2. **What about parcels you can no longer deliver on time?** If the window has already closed, should the van still take it? Remember, under our metric a late parcel scores the same as an undelivered one.
3. **Which data structure picks the next most urgent parcel?** Look at your class notes on sorting and heaps (Python's `heapq`). Which fits better, and why?
4. **Hand-check it on 4 parcels.** Make up 4 parcels with different deadlines and write out which ones your rule loads, in what order.

Step 3: Code it (2–3 h)

1. Only change the part of the copied code that decides **which parcels go first**. Leave the capacity, range and 18:00 checks as they are, or the simulator will crash.
2. Keep the function signature `def dispatch(parcel, context):` exactly the same.

3. Only use the Python standard library (`heapq`, `collections`, `math` are fine; no pandas or numpy).

Step 4: Test it

1. Test one day first: `python3 -m simulator --policy solution.policies.urgency_first --data data --day day_001 --output results/urgency_first/day_001`
2. If it crashes, read the error. It usually means a route broke a hard constraint.
3. Once day 1 works, run all days: `bash run_all.sh urgency_first`, then `python3 summarise.py`.
4. Write down the on-time rate, delivery rate and average lateness next to the baseline's.

Step 5: Analyse it (1–2 h)

1. **Big-O:** let n = number of waiting parcels. How many operations does one `dispatch` call take? Count the sort or heap step and the loop separately. Then space: what extra lists or dicts do you build?
2. **Where does it win, and where does it lose?** Compare its results day by day against the baseline in `results/summary.csv`. Is it better on every day? Which day is worst, and why?
3. **Weakness to note:** does it still zigzag across town? That's the gap Person B's policy targets.

Output by Fri 6 Nov: `urgency_first.py` working, `results/urgency_first/` with 20 days, and one page of notes covering Steps 2 and 5.

2. Person B: location-grouping policy (Sun 1 Nov – Fri 6 Nov)

The idea: the baseline zigzags across town, so each van only manages about 2 trips a day. Person B's policy should fill each van with parcels going to **the same or nearby places**, so trips are shorter and vans can do more of them. Your file: `solution/policies/location_grouping.py`.

Step 1: Create the file (10 min)

1. In `solution/policies/`, copy `baseline.py` and rename the copy `location_grouping.py`.
2. Run it unchanged: `bash run_all.sh location_grouping`. It should score the same as your baseline.

Step 2: Look at the data first (30 min)

1. Open `data/dist.csv`. There are 36 locations (L001–L036) plus HUB. Which locations are far from HUB, and which are near?
2. Open `results/baseline/day_001/log.txt` and pick one trip. Using `dist.csv`, add up how many km it drove. You'll find it goes back and forth. Note this as an example for the report.

Step 3: Design it on paper (1–2 h, no coding)

1. **Which location should the van go to first?** The one with the most parcels waiting? The nearest one? The one with the biggest total size? Pick one rule and write down why.
2. **After the first location, what next?** Fill up on that location, then add parcels from the **nearest** other location? How do you decide the visiting order?
3. **Which data structure groups parcels by location?** Look at your class notes on dictionaries and hash maps. What would the keys and values be?
4. **Which algorithm decides the stop order?** Think about the greedy algorithms from class.
5. **Hand-check it on 6 parcels:** 3 to one location, 2 to another, 1 far away. Write out the route your rule produces and its total km.

Step 4: Code it (2–3 h)

1. Change only how parcels are chosen and ordered. Keep the capacity, range and 18:00 checks as they are.
2. Keep `def dispatch(parcel, context):` exactly the same. Get distances from `context["travel_distance"][from][to]`. Never open `dist.csv` inside your policy, because the brief forbids it.
3. Standard library only.

Step 5: Test it

Same as Person A: test day_001 first, then run `bash run_all.sh location_grouping`, then `python3 summarise.py`. Also check the `trips` number: it should be well above the baseline's ~21 a day. If it isn't, the grouping isn't shortening trips.

Step 6: Analyse it (1–2 h)

1. **Big-O:** let n = waiting parcels and L = number of locations (36). How long does it take to build the groups? How long to pick the stop order?

2. **Where does it win, and where does it lose?** It probably delivers many more parcels (delivery rate up), but are they on time? Compare delivery rate against on-time rate.
3. **Weakness to note:** it ignores deadlines. A parcel due at 10:00 at a quiet location might wait all day. That's the gap Person A's policy targets.

Output by Fri 6 Nov: `location_grouping.py` working, `results/location_grouping/` with 20 days, and one page of notes covering Steps 3 and 6.

3. Together: combined policy + helpers (Sat 7 – Mon 9 Nov)

The combined policy usually wins, because each solo policy fixes only one of the baseline's two problems. Your file: `solution/policies/heuristic_best.py`, the name the brief uses as its example.

Step 1: Sat 7 Nov check-in call (1 h)

1. Each person runs `python3 summarise.py` and shares the table.
2. Each person explains their policy to the other in 5 minutes, without looking at the code. If you can't, you're not ready for the exam question.
3. Compare results: which policy has the higher on-time rate? The higher delivery rate? On which days does each one lose?

Step 2: Move shared code into `solution/policies/helpers.py` (Person B, 1 h)

1. Any function both policies use (for example: "get waiting parcels", "check whether a route breaks capacity, range or 18:00", "total route distance") goes into `helpers.py`.
2. In each policy, import it: `from solution.policies.helpers import check_route` (use your own function names).
3. Re-run both policies with `run_all.sh` and check their scores haven't changed. Same score = refactor was safe.

Step 3: Design the combined rule together (1 h, on paper)

One question to answer: **how do you balance urgency against closeness?** A few directions to discuss (pick one or invent your own):

- Pick the most urgent parcel first, then fill the van with parcels near it.
- Pick the busiest location first, but skip parcels whose deadline you can't make anyway.
- Give every parcel a score that mixes deadline and distance.

Whichever you pick, hand-check it on 6 parcels like before.

Step 4: Code and test (Person A codes, Person B tests, 3 h)

1. Build `heuristic_best.py` using the helpers.
2. Run `bash run_all.sh heuristic_best`, then `python3 summarise.py`.
3. Try 2–3 small variations (for example a different tie-breaker). Save each as its own file, such as `heuristic_best_v2.py`. The brief rewards "extensive experiments", and failed variations count, as long as you explain why they failed.

Step 5: Complexity of the combined policy (Person B)

Same method as Steps 5–6 above. It will usually be slower than either solo policy, so say whether the extra time matters. For example: one call handles about 800 parcels, so even $O(n^2)$ is under a second.

4. Report + README split (Tue 10 – Thu 12 Nov)

The report is 8 pages max, excluding references. Write in a shared Google Doc and use one heading per section below.

Report section	Weight	Owner	What to write	Length
1. Problem statement + motivation	10%	A	The delivery problem; why on-time rate matters in last-mile; the exact formula (on-time ÷ all parcels); assumptions (early = not on time; undelivered = fail); why it can't be gamed (dispatching nothing = 0%); baseline = FIFO, 27.4%	1 page

Report section	Weight	Owner	What to write	Length
2. Algorithmic approaches	20%	A writes urgency + combined, B writes grouping	Per policy: the idea in 1 sentence, the data structure + algorithm used (and that they're from class), steps in plain words or pseudocode, the 4–6 parcel hand example from your notes	2 pages
3. Complexity analysis	20%	Each writes their own, B writes combined	Per policy: time and space Big-O for one `dispatch` call, with the line-by-line reasoning; a small table comparing all policies	1.5 pages
4. Experimental results	20%	B	Table: mean on-time rate, delivery rate, avg lateness, trips per policy (from `summary.csv`); one chart of on-time rate per day per policy; explain WHY each wins or loses using the supporting metrics; mention failed variations	2 pages
5. Conclusion	10%	A	Recommended policy + why; its limitations; what you'd try with more time	0.5 page
6. References	(10% with code)	B	Course notes, any sources, and **every AI prompt and response** (paste this chat and any Gem chats)	not counted

README.md (Person B, 30 min). It's at the project root and is currently empty. It must let the marker reproduce everything. Include:

1. Python version (3.9+) and that no extra packages are needed.
2. How to run one policy on one day (the `python3 -m simulator ...` command).
3. How to run the performance monitor on one log.
4. The shortcut: `bash run_all.sh <policy>` for each policy you submit, then `python3 summarise.py`.
5. A list of each policy file and its one-line description.

Swap and proofread on Thu 12 Nov: A reads B's sections and B reads A's. Each person should be able to explain every section, since that's the exam prep.

5. Timeline + submission checklist

Submit by **Fri 13 Nov, 23:59**. Aim to finish on Thu 12 Nov so you have a day of buffer.

Dates	Person A	Person B
Sat 31 Oct	Setup + write the scorer (Section 0, Steps 1–3)	run_all.sh + summarise.py, run baseline (Section 0, Steps 4–6)
Sun 1 – Fri 6 Nov	Urgency-first policy (Section 1)	Location-grouping policy (Section 2)
Sat 7 Nov	Check-in call: share results, explain policies	Check-in call
Sat 7 – Mon 9 Nov	Code combined policy + variations	helpers.py refactor, test combined policy, its Big-O
Tue 10 – Wed 11 Nov	Report sections 1, 2, 5	Report sections 3, 4, 6 + README
Thu 12 Nov	Proofread B's sections	Proofread A's sections
Fri 13 Nov	Final check + submit	Final check

Final checklist before zipping

- [ ] `solution/policies/` has every policy you tried + `helpers.py`
- [ ] `baseline.py` and all `__init__.py` files are unchanged

- [ ] `simulator/` and `data/` are unchanged
- [ ] `results/<policy>/day_001 … day_020/` has both `log.txt` and `metrics.json`, for **every** policy (including baseline)
- [ ] Deleted any old results from versions of a policy you later changed, then re-ran it, so results match the code
- [ ] `README.md` commands copy-pasted into a fresh terminal and they work
- [ ] Report is 8 pages or fewer (excluding references), with all AI prompts and responses in references
- [ ] Report numbers match `results/summary.csv`
- [ ] Got the AI-use rule in writing from Prof Tan (the PDF still says Gemini Gem only)
- [ ] Zip the whole `project/` folder and upload to eLearn